Acelerador MAC/MAS Pipeline en bfloat16 usando OpenLane SKY130

Ana Sofia Giraldo, Jorge Esteban Ruiz, Luisa Fernanda Vélez, Maria José Castañeda
Ingeniería Electrónica
Universidad Tecnológica de Pereira
Pereira, Colombia
anasofia.giraldo@utp.edu.co, jorge.ruiz1@utp.edu.co, luisafernanda.velez@utp.edu.co, mariajose.castaneda@utp.edu.co

***Abstract*—En este proyecto se presenta la implementación ASIC de un acelerador MAC/MAS pipeline utilizando el formato de punto flotante bfloat16. El diseño fue desarrollado en Verilog HDL e implementado mediante el flujo OpenLane utilizando la librería estándar SKY130. La arquitectura soporta operaciones Multiply-Accumulate (MAC) y Multiply-Accumulate-Subtract (MAS) usando una estructura pipeline optimizada para desempeño temporal. La verificación funcional fue realizada utilizando Icarus Verilog y GTKWave, mientras que la implementación física incluyó síntesis, placement, routing, análisis STA y generación del GDS final.**

***Index Terms*—ASIC, OpenLane, SKY130, MAC, MAS, Pipeline, Punto Flotante, bfloat16**

I. Introducción

Las operaciones Multiply-Accumulate (MAC) representan uno de los bloques fundamentales en arquitecturas de procesamiento digital moderno, especialmente en aplicaciones relacionadas con procesamiento digital de señales (DSP), aceleradores de inteligencia artificial, redes neuronales y sistemas embebidos de alto desempeño. Debido a que múltiples algoritmos matemáticos requieren operaciones repetitivas de multiplicación y acumulación, los módulos MAC son ampliamente utilizados para optimizar velocidad y eficiencia computacional.

En este proyecto se desarrolló un acelerador MAC/MAS pipeline utilizando el formato de punto flotante *bfloat16*, implementado completamente en Verilog HDL y posteriormente sintetizado como un circuito ASIC mediante el flujo OpenLane utilizando la tecnología SKY130.

El formato *bfloat16* fue seleccionado debido a su uso creciente en aplicaciones de inteligencia artificial y aceleradores de hardware, ya que permite reducir complejidad de hardware y consumo de recursos manteniendo un rango dinámico adecuado para operaciones numéricas de precisión moderada.

El objetivo principal del proyecto fue diseñar una arquitectura pipeline capaz de ejecutar operaciones de multiplicación y acumulación en punto flotante, optimizando el desempeño temporal del circuito y permitiendo su implementación física mediante herramientas de diseño ASIC open-source.

El acelerador implementado soporta dos operaciones principales:

- Operación MAC:

$$ACC = ACC + (A \times B) \tag{1}$$

- Operación MAS:

$$ACC = ACC - (A \times B) \tag{2}$$

Para mejorar el desempeño y reducir el retardo de la ruta crítica, la arquitectura fue dividida en múltiples etapas pipeline, permitiendo sincronizar adecuadamente las operaciones internas de multiplicación, suma/resta y actualización del acumulador.

Finalmente, el diseño fue verificado funcionalmente mediante simulaciones comportamentales utilizando Icarus Verilog y GTKWave, y posteriormente implementado físicamente utilizando OpenLane, incluyendo síntesis lógica, placement, routing, análisis STA y generación del layout final.

II. Arquitectura

El diseño desarrollado corresponde a un acelerador MAC/-MAS pipeline implementado en Verilog HDL. El módulo principal del sistema es `mac_mas_top.v`, encargado de coordinar las operaciones de multiplicación, suma/resta y acumulación utilizando datos en formato de punto flotante *bfloat16*.

La arquitectura fue diseñada utilizando una organización pipeline con el objetivo de mejorar el desempeño temporal del circuito y reducir la longitud de la ruta crítica. Para ello, las operaciones internas fueron divididas en múltiples etapas secuenciales sincronizadas mediante señales de control y registros intermedios.

La estructura general del sistema está compuesta por los siguientes bloques funcionales:

- **Multiplicador de punto flotante**: Encargado de realizar la operación de multiplicación entre las entradas `A` y `B` utilizando el formato bfloat16.
- **Sumador/restador de punto flotante**: Realiza la operación de suma o resta entre el resultado del multiplicador y el valor almacenado en el acumulador.

- **Registros pipeline**: Permiten sincronizar las diferentes etapas internas del circuito y mantener estabilidad temporal entre operaciones consecutivas.
- **Registro acumulador**: Almacena el resultado parcial o final de las operaciones MAC/MAS realizadas por el sistema.
- **Lógica de control**: Controla la operación ejecutada mediante la señal `opcode`, además de coordinar las señales de habilitación y sincronización del pipeline.

Las operaciones implementadas por el acelerador son seleccionadas mediante la entrada `opcode`. La Tabla reftab:opcode muestra la codificación utilizada para cada operación.

TABLE I
CODIFICACIÓN DE OPERACIONES

Opcode	Operación
0011	MAC
1011	MAS

A. Operación MAC

La operación MAC (Multiply-Accumulate) realiza primero la multiplicación entre las entradas `A` y `B`, y posteriormente suma el resultado al valor actual almacenado en el acumulador:

$$ACC = ACC + (A \times B) \tag{3}$$

Esta operación es ampliamente utilizada en algoritmos DSP y redes neuronales debido a la alta cantidad de operaciones repetitivas de multiplicación y acumulación presentes en dichos sistemas.

B. Operación MAS

La operación MAS (Multiply-Accumulate-Subtract) realiza la multiplicación entre las entradas y posteriormente resta el resultado al valor almacenado en el acumulador:

$$ACC = ACC - (A \times B) \tag{4}$$

Esta funcionalidad permite ampliar las capacidades aritméticas del acelerador y soportar operaciones matemáticas adicionales utilizadas en diferentes algoritmos digitales.

C. Estructura Pipeline

La arquitectura implementa una organización pipeline para aumentar el throughput del sistema y mejorar el cierre temporal durante la implementación física del ASIC.

Las principales etapas pipeline del diseño son:

1) **Etapa de multiplicación**: Se calcula el producto de las entradas `A` y `B` utilizando el multiplicador de punto flotante.
2) **Etapa de suma/resta**: El resultado de la multiplicación es enviado al módulo sumador/restador junto con el valor almacenado en el acumulador.
3) **Etapa de actualización del acumulador**: El resultado final es almacenado nuevamente en el registro acumulador para futuras operaciones.

El flujo general de datos dentro del acelerador puede representarse mediante el siguiente diagrama funcional:

$$\texttt{A,B} \rightarrow \texttt{MULT} \rightarrow \texttt{ADD/SUB} \rightarrow \texttt{ACC}$$

La utilización de etapas pipeline permitió reducir el impacto de la ruta crítica y facilitar la implementación física del diseño utilizando el flujo OpenLane y la tecnología SKY130.

III. DISEÑO RTL

El diseño RTL del acelerador fue desarrollado completamente en Verilog HDL utilizando una arquitectura modular. Cada bloque funcional fue implementado de manera independiente con el objetivo de facilitar la depuración, reutilización de código y escalabilidad del sistema.

La arquitectura RTL fue diseñada para soportar operaciones aritméticas de punto flotante en formato *bfloat16*, utilizando una organización pipeline que permite mejorar el desempeño temporal del circuito y reducir la longitud de la ruta crítica.

Los principales módulos implementados en el diseño son:

- `fpmul.v`: Implementa el multiplicador de punto flotante encargado de calcular el producto entre las entradas `A` y `B`.
- `fp16sum_res_pipe.v`: Implementa el sumador/restador pipeline de punto flotante utilizado para realizar las operaciones MAC y MAS.
- `mac_mas_top.v`: Corresponde al módulo principal del sistema. Este bloque integra todas las etapas del pipeline, controla la lógica de operación y administra el acumulador interno.
- `rounder.v`: Encargado de realizar el redondeo de resultados durante las operaciones de punto flotante.
- `myreg.v`: Implementa registros utilizados para sincronización pipeline y almacenamiento temporal de señales internas.

El módulo principal `mac_mas_top.v` coordina el flujo completo de datos dentro del acelerador. Inicialmente, las entradas son procesadas por el multiplicador de punto flotante. Posteriormente, el resultado es enviado al bloque sumador/restador junto con el valor almacenado previamente en el acumulador. Finalmente, el resultado obtenido es almacenado nuevamente para futuras operaciones.

La señal `opcode` permite seleccionar dinámicamente la operación a ejecutar, mientras que las señales `en` y `ready` son utilizadas para controlar la sincronización del pipeline y validar la disponibilidad de resultados.

La Figura 1 muestra el esquemático RTL generado utilizando Yosys y visualizado mediante netlistsvg. En esta representación es posible observar la conexión entre las diferentes etapas pipeline, los registros internos y los módulos aritméticos utilizados en el diseño.

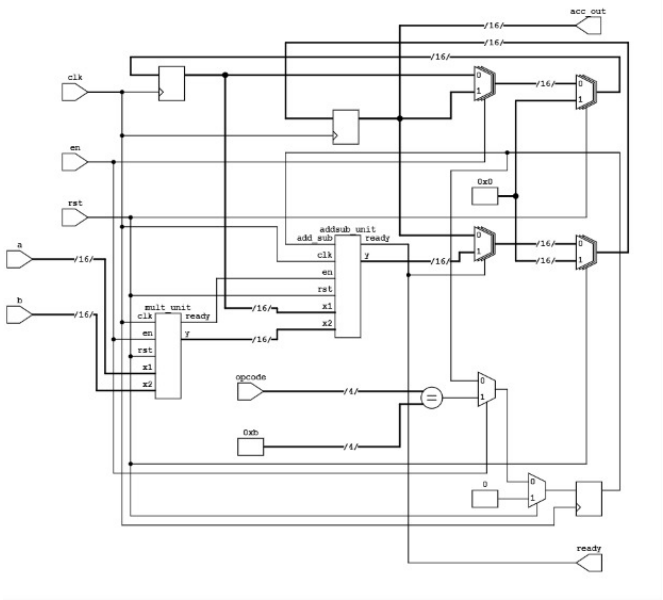

Fig. 1. Esquemático RTL generado utilizando netlistsvg

IV. Verificación Funcional

La verificación funcional del acelerador fue realizada utilizando Icarus Verilog como simulador y GTKWave para la visualización de señales temporales.

Para validar el correcto funcionamiento del diseño se desarrolló un testbench dedicado capaz de generar diferentes estímulos de entrada y verificar automáticamente las operaciones implementadas por el sistema.

Las principales funcionalidades verificadas fueron:

- Correcta ejecución de la operación MAC
- Correcta ejecución de la operación MAS
- Funcionamiento del acumulador interno
- Sincronización entre las etapas pipeline
- Temporización de la señal `ready`
- Actualización correcta de resultados post-pipeline

Durante la simulación se utilizaron operandos en formato *bfloat16*, específicamente el valor hexadecimal `3F80`, correspondiente al número decimal 1.0.

En la operación MAC, el sistema realiza:

$$ACC = 0 + (1.0 \times 1.0) \tag{5}$$

obteniendo como resultado:

$$ACC = 1.0 \tag{6}$$

representado en formato hexadecimal como:

$$3F80 \tag{7}$$

Posteriormente, en la operación MAS, el sistema ejecuta:

$$ACC = 1.0 - (1.0 \times 1.0) \tag{8}$$

obteniendo nuevamente un acumulador igual a cero:

$$ACC = 0 \tag{9}$$

representado hexadecimalmente como:

$$0000 \tag{10}$$

Los resultados obtenidos durante la simulación se resumen en la Tabla II.

TABLE II
Resultados de Simulación

Operación	Resultado
MAC	3F80
MAS	0000

La Figura 2 muestra las formas de onda obtenidas durante la simulación funcional en GTKWave. En ellas es posible observar la activación de las señales de control, el comportamiento del pipeline y la actualización del acumulador interno.

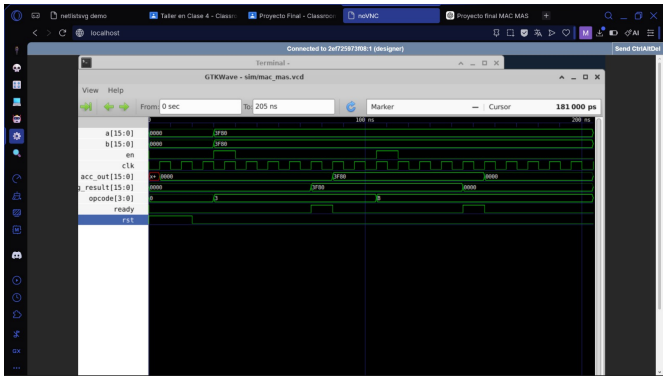

Fig. 2. Waveform de simulación funcional en GTKWave

V. Flujo ASIC con OpenLane

La implementación física del acelerador fue realizada utilizando el flujo ASIC OpenLane junto con el PDK SKY130 de SkyWater.

OpenLane proporciona una cadena automatizada de herramientas open-source para síntesis lógica, implementación física, routing y análisis temporal de circuitos integrados.

El flujo utilizado en este proyecto incluyó las siguientes etapas:

1) **Síntesis lógica**: Conversión del diseño RTL en una netlist compuesta por celdas estándar de la librería SKY130.
2) **Floorplanning**: Definición inicial de la distribución física del diseño y organización de las áreas internas del chip.
3) **Placement**: Ubicación física de las celdas estándar dentro del área del diseño.
4) **Clock Tree Synthesis (CTS)**: Inserción y balanceo de la red de reloj para minimizar skew y mejorar sincronización.
5) **Routing**: Interconexión física de todas las señales del diseño utilizando capas metálicas.
6) **Static Timing Analysis (STA)**: Verificación temporal post-routing para identificar posibles violaciones de setup y hold.
7) **Generación del GDSII**: Creación del archivo final utilizado para fabricación física del circuito integrado.

Para mejorar el desempeño temporal del diseño se utilizó la estrategia de optimización `DELAY`, orientada a reducir retardos críticos y maximizar la frecuencia de operación alcanzable.

VI. Layout Físico

Después de completar exitosamente el flujo de implementación física, OpenLane generó el layout final del acelerador MAC/-MAS.

El layout representa físicamente la distribución de celdas estándar, interconexiones metálicas, buffers de reloj y rutas internas del diseño dentro del área total del chip.

La Figura 3 muestra el layout físico final obtenido después de las etapas de placement y routing.

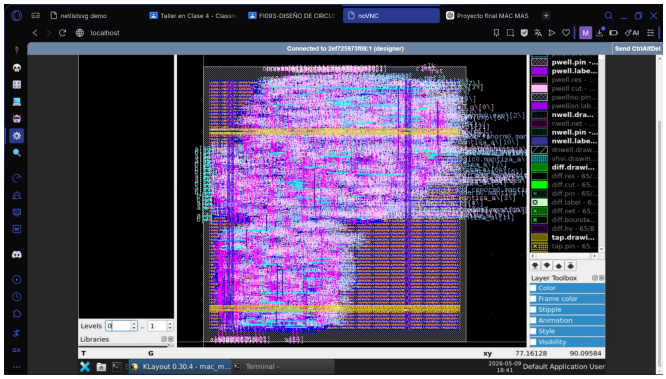

Fig. 3. Layout físico final generado utilizando OpenLane

El diseño implementado logró completar correctamente todas las etapas físicas requeridas para la generación del archivo GDSII final.

VII. Análisis Temporal

El análisis temporal estático (STA) fue realizado después de finalizar el routing utilizando OpenROAD dentro del flujo OpenLane.

El objetivo principal de esta etapa fue verificar que todas las rutas críticas del diseño cumplieran las restricciones temporales establecidas para el reloj del sistema.

Los resultados mostraron que el diseño completó exitosamente las verificaciones físicas principales:

- Cero errores DRC
- Cero errores LVS
- Sin violaciones hold

Sin embargo, se observaron algunas violaciones de setup relacionadas con la profundidad pipeline y la complejidad de las operaciones de punto flotante implementadas.

El peor valor de setup slack obtenido fue:

$$WNS = -0.9157ns \tag{11}$$

considerando un periodo objetivo de reloj igual a:

$$T_{clk} = 10ns \tag{12}$$

A partir de este resultado puede estimarse la frecuencia máxima aproximada de operación mediante:

$$f_{max} \approx \frac{1}{10ns + 0.9157ns} \tag{13}$$

obteniendo finalmente:

$$f_{max} \approx 91.6MHz \tag{14}$$

Estos resultados demuestran que el pipeline permitió alcanzar una frecuencia relativamente alta considerando la complejidad aritmética del diseño y las limitaciones tecnológicas del nodo SKY130.

VIII. Resultados de Área y Potencia

Las métricas finales obtenidas después del flujo post-layout permiten evaluar el desempeño físico y energético del acelerador implementado.

La Tabla III resume las principales métricas obtenidas durante la implementación ASIC.

TABLE III
MÉTRICAS ASIC FINALES

Métrica	Valor
Área total	21205.3 μm^2
Número de instancias	3128
Potencia estimada	4.6 mW
Frecuencia máxima estimada	91.6 MHz
Errores DRC	0
Errores LVS	0

Los resultados obtenidos evidencian que el diseño logró completar satisfactoriamente la implementación física manteniendo un consumo de potencia moderado y un área razonable para un acelerador aritmético pipeline de punto flotante.

IX. Problemas Encontrados y Depuración

Durante el desarrollo del proyecto se identificaron diversos problemas relacionados principalmente con sincronización pipeline y temporización de señales internas.

Uno de los principales desafíos encontrados fue la alineación temporal entre las etapas de multiplicación y suma/resta. Debido a la latencia interna de los módulos pipeline, inicialmente el acumulador era actualizado antes de que los resultados válidos estuvieran disponibles.

Adicionalmente, se presentaron problemas relacionados con:

- Sincronización del acumulador interno
- Temporización incorrecta de la señal `ready`
- Manejo de latencia pipeline
- Alineación temporal entre módulos aritméticos
- Captura prematura de resultados

Estos problemas fueron solucionados mediante:

- Inserción de registros adicionales de sincronización
- Corrección de la lógica de actualización del acumulador
- Ajuste del control de la señal `ready`
- Reorganización temporal de las etapas pipeline

Después de las correcciones implementadas, el diseño logró producir resultados funcionalmente correctos tanto en simulación RTL como en simulación post-layout.

X. Conclusiones

En este proyecto se implementó exitosamente un acelerador MAC/MAS pipeline utilizando operaciones de punto flotante en formato *bfloat16* y el flujo ASIC OpenLane basado en la tecnología SKY130.

El diseño logró completar correctamente todas las etapas del flujo de implementación física, incluyendo síntesis lógica, placement, routing, análisis temporal y generación del archivo GDSII final.

La utilización de arquitectura pipeline permitió mejorar significativamente el desempeño temporal del sistema y facilitar el cierre temporal durante la implementación ASIC.

Asimismo, las verificaciones funcionales realizadas mediante Icarus Verilog y GTKWave confirmaron el correcto funcionamiento de las operaciones MAC y MAS implementadas.

Finalmente, el diseño pasó exitosamente las verificaciones físicas DRC y LVS, validando la consistencia y correctitud de la implementación física obtenida.

References

[1] OpenLane Documentation. [Online]. Available: https://openlane.readthedocs.io

[2] Google SKY130 PDK Documentation. [Online]. Available: https://skywater-pdk.readthedocs.io

[3] Yosys Open Synthesis Suite. [Online]. Available: https://yosyshq.net/yosys/